Laboratory Experiment – 6

Aim: To construct and implement a 4-bit shared bus system using multiplexers and parallel-load registers, and to examine the mechanism of transferring data between several registers over a common communication path.

Theory:

In a digital system, a common bus provides a single shared pathway through which data is routed between different components such as registers, memory units, and processing elements. Rather than establishing a dedicated wire between every pair of components (which grows impractically large), a bus architecture reduces hardware complexity by allowing all components to share the same set of signal lines.

a) Concept of a Bus:

A bus is formed by grouping multiple parallel signal lines together. In the case of a 4-bit bus, four parallel wires are used, each carrying one bit at a time. At any given clock cycle, the bus can transport a complete 4-bit value from one source to any destination connected to it.

b) Role of Multiplexers in Bus Construction:

A multiplexer (MUX) acts as a selector: it accepts several input lines and routes exactly one of them to the output, based on control signals known as select lines. In bus design, one MUX is required per bit-position of the bus.

For a 4-bit bus connecting four registers:

- A total of four 4-to-1 multiplexers are required (one per bit).
- Each MUX receives one bit from each of the four registers as its inputs.
- A common pair of select lines (S1, S0) determines which register drives the bus at any time.

Since all four MUXes share identical select lines, the same register is simultaneously selected across all four bit positions, ensuring correct 4-bit data transfer.

Illustration:

Consider registers R0, R1, R2, and R3, each 4-bits wide:

- S1=0, S0=0 → R0 drives the bus
- S1=0, S0=1 → R1 drives the bus
- S1=1, S0=0 → R2 drives the bus
- S1=1, S0=1 → R3 drives the bus

c) Register Operation:

Each register in this design is a parallel-load, 4-bit register equipped with a Write Enable (WE) signal. Registers perform two roles in the bus system:

- Acting as a data source: the register's stored value is fed as input to one of the MUX inputs.
- Acting as a data destination: when WE is asserted and a clock edge arrives, the register captures whatever value is currently on the bus.

The separation of source selection (via MUX select lines) from destination selection (via individual WE signals) allows fine-grained control over each transfer.

d) Data Transfer Procedure:

Every register-to-register transfer is accomplished in two coordinated steps:

- Step 1 – Source Selection: Set the MUX select lines (S1, S0) to the binary code corresponding to the source register. The bus now carries the output of that register.
- Step 2 – Destination Load: Assert the WE (Write Enable) signal of the target register. On the next rising clock edge, the destination register latches the value from the bus.

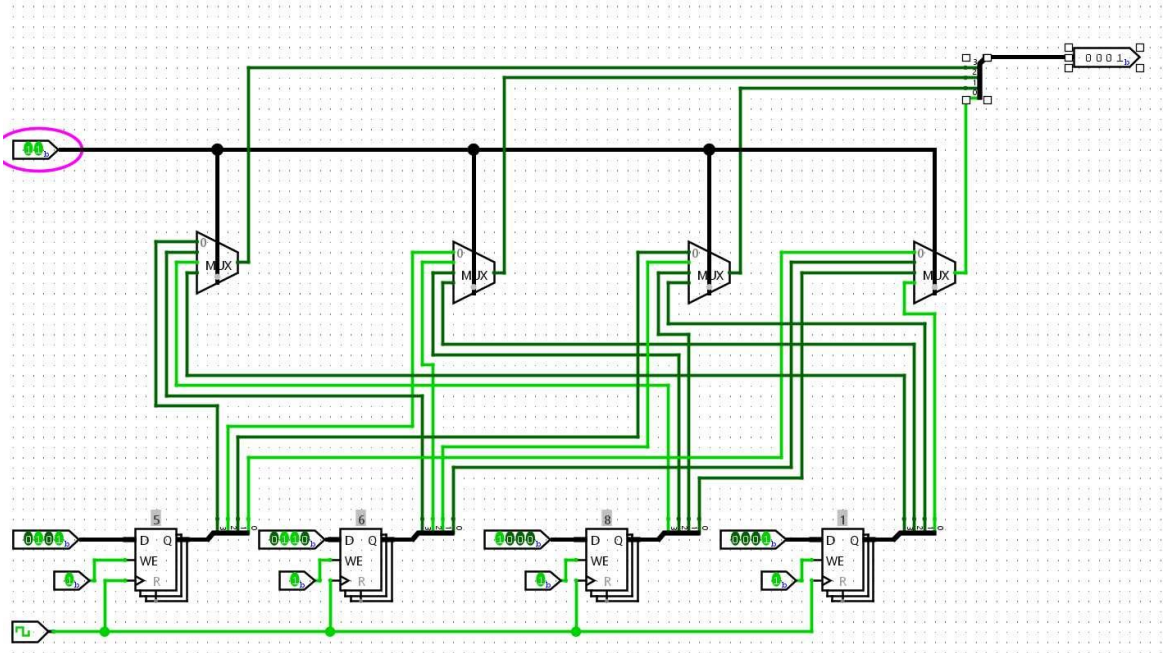
Worked Example – Transferring R1 → R3:

- Set S1=0, S0=1 so that R1's data is routed onto the bus.
- Assert WE of R3 and apply a clock pulse → R3 now holds the previous contents of R1.

e) Advantages of the Common Bus Architecture:

- Significantly reduces the number of interconnect wires compared to point-to-point connections.
- Enables flexible and scalable data routing as the number of registers grows.
- Centralized control logic simplifies system design and timing management.
- Supports easy expansion: adding a register requires only an additional MUX input, not a complete rewiring.

Logisim Circuit Implementation:



Conclusion:

This experiment successfully demonstrated the design and simulation of a 4-bit common bus system using Logisim. By connecting four registers to four 4-to-1 multiplexers sharing common select lines, we achieved controlled data routing between registers. The experiment confirmed that MUX-based bus architecture effectively reduces wiring complexity while providing a flexible mechanism for inter-register data transfer, forming the foundation for more complex digital processor designs.